

M08 / PROTECTED ANSWERS

M08 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
DE08-01 Ingestion architecture: source -> landing -> staging	3
DE08-02 Defensive CSV ingestion	3
DE08-03 JSON and nested-record ingestion	3
DE08-04 Parquet ingestion and metadata inspection	4
DE08-05 REST API contracts, status codes and timeouts	4
DE08-06 API authentication, pagination and rate limits	4
DE08-07 Retries, backoff, jitter and transient failures	5
DE08-08 Database extraction and safe query boundaries	5
DE08-09 Historical batch landing and raw metadata	5
DE08-10 Incremental loads and high-water marks	6
DE08-11 CDC concepts and change-event semantics	6
DE08-12 Schema drift, compatibility, time zones and quarantine	6
DE08-13 Checkpoint/state management and replay safety	7
DE08-14 Idempotent landing and content identity	7
DE08-15 Late-arriving data, event time and processing time	7
DE08-16 Ingestion observability: counts, latency and run metadata	8
DE08-17 Ingestion integration, reconciliation and backfill proof	8

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

OPEN AFTER A SAVED GENUINE ATTEMPT

Standalone Zero-to-Job-Ready Data Engineer 2026

Open only after a saved genuine attempt. Read the weak lesson review, close it, then solve a changed retry.

DE08-01 | Ingestion architecture: source -> landing -> staging

Strong evidence separates immutable raw landing from rebuildable staging, keeps quarantine with reasons, and commits state only after complete success. A retry creates new run evidence rather than overwriting yesterday.

Worked repair: Draw four boxes: source, landing, staging and quarantine, then add a separate state/checkpoint box. The source is what you read; landing is the immutable evidence of what arrived; staging is rebuildable parsed/validated data; quarantine keeps rejected evidence. State is not "the last row I saw" - it is the last position whose downstream effects were durably accepted. If a failed run overwrote yesterday's landing or advanced state despite incomplete staging, repair ownership before touching transformation code. Re-run with a new run ID and prove the prior evidence still exists.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-02 | Defensive CSV ingestion

Good CSV: 4 source rows -> 4 accepted, 0 quarantine. Bad fixture must preserve raw bytes and classify defects; accepted+quarantine must equal source rows. Leading-zero IDs stay text.

Worked repair: Diagnose parsing and validation separately. First use a real CSV parser and prove a quoted comma stays inside one field and a leading-zero ID remains text. Then validate business rules without destroying the raw values. If four source rows become one accepted + two quarantined, one row disappeared; stop immediately because $1 + 2 \neq 4$. If a row has several defects, preserving all useful reasons is better than silently keeping only the final coercion error. The repair is complete only when raw count = accepted + quarantined and accepted keys satisfy the declared uniqueness rule.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-03 | JSON and nested-record ingestion

J3001 produces exactly two item-grain rows. State the key (order_id,line_no) for this fixture and reconcile item totals from source versus flattened rows.

Worked repair: State the grain before flattening. The parent JSON document is not automatically the output grain; for the supplied nested order the item table is one row per (order_id, line_no). Copy parent identifiers deliberately onto each child row and preserve enough source identity to trace back. If J3001 has two item objects, exactly two item-grain rows should emerge. A common wrong result is to duplicate the parent order total on both child rows and later sum it twice. Validate child count, composite-key uniqueness and one independently recomputed item total.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-04 | Parquet ingestion and metadata inspection

Parquet logical row count and keys match source, schema is inspected, projected read contains only requested columns, and metadata row-group evidence is saved. Native pyarrow execution is required on learner PC.

Worked repair: Separate logical equivalence from physical-layout evidence. The Parquet conversion should preserve row/key/value controls from its source. Then inspect schema and metadata, and issue a projected read that requests only named columns. Row-group statistics can make predicate skipping possible, but a tiny learner file cannot prove a production performance gain. If PyArrow is unavailable, do not fabricate a screenshot or timing result: save the exact command/script and run the native metadata/projection check on the learner PC before claiming runtime mastery.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-05 | REST API contracts, status codes and timeouts

/health=200; unauthenticated order endpoint=401; authenticated valid call=200; missing path=404. JSON decode is never used as proof of HTTP success and timeout is explicit.

Worked repair: Classify failures in order: transport, HTTP status, media type, parse, payload contract. A 200 response with text/plain is not successful JSON ingestion, while a 404 is not a JSON-decoding problem. Make the client apply an explicit timeout, check status policy, verify expected content type, parse JSON, then verify required fields/types. If your code calls .json() before checking status/content type, move parsing later. Save at least one controlled 401/404 or wrong-content response so the difference is visible rather than memorized.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-06 | API authentication, pagination and rate limits

page_size=3 yields all 10 unique v1 IDs; one page-2 429 is retried according to Retry-After; 401 is not repeatedly retried; token is absent from saved logs.

Worked repair: Pagination completeness is a set/ordering problem, not "I reached page 4." Collect IDs across pages, prove the count equals the API-declared total for the fixture, and prove

uniqueness. For `page_size=3` the supplied `v1` route must still yield all ten unique IDs. When page 2 returns one 429, respect the bounded retry policy and `Retry-After`; when credentials are wrong and return 401, fail rather than hammering the service. Finally inspect saved logs and evidence to prove the bearer token itself was not recorded.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-07 | Retries, backoff, jitter and transient failures

One-time 500 recovers under bounded retry. 429 may retry. Wrong auth fails quickly. Attempt count and backoff are visible. Write retries are not assumed safe without idempotency semantics.

Worked repair: Build a small retry decision table: timeout/connection error and selected 5xx may be transient; 429 may retry according to server guidance; bad authentication or a deterministic schema defect should not be retried blindly. Add a maximum-attempt budget and visible attempt count. If the one-time 500 succeeds on the next allowed attempt, that proves bounded transient recovery. Then change the request into a side-effecting write and explain why retry safety now depends on an idempotency key or server transaction semantics, not merely on exponential backoff.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-08 | Database extraction and safe query boundaries

After (08:15,D5002) the local extract returns D5003 then D5004. Query uses explicit columns, parameters and deterministic order. Read-only connection rejects mutation.

Worked repair: Keep the database query bounded and deterministic. Use named parameters for the composite watermark and order by the same tuple used by the boundary: (`updated_at`, `order_id`). From the supplied checkpoint (08:15,D5002), the expected next rows are D5003 then D5004. If the same timestamp can occur twice but the query filters only `updated_at > :ts`, it can skip a tied row. Also prove the learning connection cannot mutate source data when it is intended to be read-only; a successful `SELECT` does not prove least privilege.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-09 | Historical batch landing and raw metadata

Both historical files land in one named run, each manifest entry records original name/bytes/SHA256, and a second run does not overwrite first-run evidence.

Worked repair: Treat historical landing as an evidence operation. For every source file keep original name, retrieval/run identity, byte size and SHA-256 in the manifest before transformation. Run the landing process twice. The second run should create distinct run evidence rather than overwrite or rename the first raw copy into invisibility. A checksum proves

byte identity, not business correctness, so keep row-count and later contract validation separate. If you cannot answer "which exact bytes produced this staging batch?", lineage is incomplete.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-10 | Incremental loads and high-water marks

Initial v1 fetch returns 10 unique records and a candidate final state (2026-08-05T09:20:00Z,A4010). Before durable output exists, the live checkpoint must remain absent/unchanged. Persist the ten rows, validate identity/counts, then atomically commit the candidate checkpoint. A4007/A4008 tie proves timestamp-only watermarks unsafe; a no-new-data rerun then returns zero.

Worked repair: If state advanced before output, remove checkpoint mutation from the fetch function. Have fetch return (rows, candidate_checkpoint). Make the caller persist/validate output first, then call a separate commit_checkpoint. Inject a failure between fetch and persist: prior state must remain unchanged. Inject a failure after persist but before commit: replay may reread rows, so staging must be idempotent, but it must not skip them. The A4007/A4008 tied timestamp is the proof case for why (timestamp,id) is safer than timestamp alone.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-11 | CDC concepts and change-event semantics

Applying ordered CDC events leaves A5001 status paid and removes A5002 under current-state semantics while preserving event history. Duplicate event ID does not duplicate effect.

Worked repair: Replay the supplied CDC events in deterministic event order and separate event history from current state. An insert/update/delete event is evidence of a change, while the current-state table is the reduced result after those events. If A5001 finishes as paid and A5002 is removed under the

current-state contract, keep the original event records so that result is explainable. Apply the same event ID twice and verify its business effect is not duplicated. If event identity is missing, state what additional source contract is needed before promising exactly-once effect.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-12 | Schema drift, compatibility, time zones and quarantine

v2 raw page lands but contributes zero v1 staging rows; records are quarantined as unsupported/incompatible and live v1 checkpoint does not advance. Offset-less time is not silently assigned local time. A trailing Z alone is not proof of a valid timestamp: parse the ISO-8601 value and reject malformed/impossible values such as not-a-dateZ.

Worked repair: Validate schema/version and timestamp syntax before promoting a row to trusted staging. The v2 payload can be landed intact even when the v1 staging contract rejects

it; that preserves evidence without pretending compatibility. Parse timestamps rather than checking string suffixes - not-a-dateZ ends in Z but is not a timestamp. Decide how offsets are normalized and never silently assign a timezone to an offset-less value. If an incompatible page is quarantined, the v1 live checkpoint must not move past it unless the product contract explicitly defines a different recovery policy.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-13 | Checkpoint/state management and replay safety

Replay creates new landing evidence but live checkpoint remains byte-identical under no-commit policy; failure before commit leaves prior state intact; checkpoint ownership is explicit.

Worked repair: Write down checkpoint ownership as a tiny state machine: prior committed state -> fetch candidate -> persist/validate output -> atomically commit new state. A replay/debug run should not mutate the live production-style checkpoint unless that mode explicitly owns state. Compare checkpoint bytes before and after a no-commit replay. Then force failure before the commit and prove prior state is unchanged. If state can move backward, forward on failure, or be updated by two unrelated components, the checkpoint contract is not safe enough.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-14 | Idempotent landing and content identity

Two landings can keep identical bytes/run evidence while logical staging remains one row per business key. SHA proves byte identity, not business identity.

Worked repair: Distinguish file identity from business identity. Two landed files may have the same SHA-256 because they contain identical bytes; retaining both run manifests can still be useful audit evidence. Downstream, however, current-state staging should not create two logical copies of the same business key merely because the same raw bytes arrived twice. Prove the idempotency boundary with key counts and totals. Conversely, two different files can contain the same business key with changed values, so checksum inequality alone does not tell you whether to insert, update, reject or preserve history.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-15 | Late-arriving data, event time and processing time

L2 is late under 60-minute policy; event time is preserved. Classification changes predictably under a new tolerance and does not rewrite the business timestamp.

Worked repair: Keep event time and processing/arrival time as separate fields. Classify lateness from the policy, not from intuition. Under the supplied 60-minute tolerance, L2 is late; changing the tolerance may change classification, but it must not rewrite the event's original timestamp to

make the record look punctual. Explain which downstream operation is affected - window closure, watermark, correction/backfill, or reporting freshness - and keep enough metadata to replay the classification under a changed policy.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-16 | Ingestion observability: counts, latency and run metadata

Run metadata includes counts/retries/state/status and accepted+quarantined reconciles to source scope. Missing row with SUCCESS is detected by equation, not hidden by logs.

Worked repair: A green SUCCESS string is not an observability system. Record run ID, source identity/range, start/end time, accepted/quarantined counts, retry count, prior/candidate/committed state and final status. Reconcile source scope = accepted + quarantined where that equation applies. Then create a deliberately inconsistent metadata record - for example one missing row but status SUCCESS - and show the equation detects it. Logs answer "what happened"; metrics/reconciliation answer "did the data population remain trustworthy."

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.

DE08-17 | Ingestion integration, reconciliation and backfill proof

Integration proof includes immutable landing, contract validation, quarantine, bounded retry, composite state, replay/idempotency, late/event-time reasoning, structured run metadata, failure injection and handoff.

Worked repair: Defend the pipeline from source to handoff. Start with exact source identity, then show immutable landing, parse/schema checks, quarantine evidence, bounded retry, deterministic pagination/order, composite state, idempotent replay, event-time/late-data handling and structured run metadata. Inject at least one failure before state commit and one duplicate/replay case. The final proof is not that every command is green; it is that counts/keys/checksums/state and known failure behavior reconcile. A new engineer should be able to reproduce a backfill without guessing which files or checkpoint own the truth.

Repair questions: What did you expect? What did you actually observe? Which boundary failed - acquisition, contract, identity, state, retry, time, or reconciliation? Repair only that boundary, then rerun a changed case.